# Table of Contents

Cover page

[Institution name]

Department of Computer Science and Engineering

Operating Systems Mini Project

msh: A Minimal UNIX Shell

Course: Operating Systems (T7998), Credit 4

Year / Semester: 2026 / III

Batch: 2025–29

Faculty guide: Dr. Bibhuprasad Sahu

Academic year: 2026–27

Submitted by

[Member 1 name] [PRN / roll number]

[Member 2 name] [PRN / roll number]

[Member 3 name] [PRN / roll number]

[Member 4 name] [PRN / roll number]

Abstract

This project presents msh, a minimal UNIX shell written in POSIX C and used from the command line. A shell reads a command line and turns it into processes, pipes, and open files. msh does that work with fork, execvp, pipe, dup2, and waitpid. It does not call system(), and it does not pass the line to another shell. Commands that must change the shell, such as cd and exit, run in the shell process when they are a simple command. External commands run in children. A pipeline is a sequence of children joined by anonymous pipes and placed in one process group. On a terminal, the shell gives that group the foreground, so Ctrl+C stops

the command and returns control to msh. Background commands stay in their own process group and are reaped so they do not remain zombies. The source is split into a parser, an executor, builtins, and a job table. Thirty-six functional tests and a terminal interrupt test cover ordinary commands, pipelines, redirection, and invalid input. On the measurement machine a builtin pwd cost about 0.024 ms, an external true cost about 1.244 ms, and a two-process pipeline cost about 1.823 ms. The result is a shell small enough to explain line by line and complete enough to demonstrate process creation, file descriptors, and inter-process communication.

## 1. Introduction

Users meet an operating system through programs, and the program they meet first is usually a shell. The shell looks like a place where text runs. What it actually does is a short sequence of operating-system operations: read a line, split it into commands, create a process for each command, wire the processes together with pipes, point their standard input and output at files when asked, and wait until they finish. Those operations are the same ones a course covers under process management, file descriptors, and inter-process communication. Building the shell makes the operations visible, because each one is a system call in the source rather than a paragraph in a textbook.

UNIX made this design explicit early. Ritchie and Thompson described a system in which processes are cheap to create and in which files, devices, and pipes share one read/write interface [1]. Bourne’s shell, published in 1978, is both a command language and a programming language sitting on that interface. It redirects input and output, runs pipelines, and searches a path for programs [2]. Kernighan and Pike later taught the same model by showing how small programs become useful when a shell can connect them [3]. A student shell does not need the programming-language half of Bourne’s shell. It needs the process half: create a process, arrange its file descriptors, and collect its status.

msh is that shell. The user types a line at a prompt, or feeds the shell a script on standard input. The program parses the line into one or more commands. A simple builtin runs inside msh. Anything else is forked. The child restores the default behavior of the interrupt signal, applies pipes and redirection, and calls execvp. The parent waits, unless the line ended with an ampersand, in which case the child is recorded as a background job. When the shell is in the foreground of a terminal, the child pipeline is moved into its own process group and that

group is made the foreground group. Ctrl+C is delivered to the pipeline. The shell, which is no longer in the foreground group, keeps running.

The interface is the command line on purpose. A shell’s job is to be the command line. A graphical wrapper would hide the prompt, the exit status, and the interrupt, which are the behaviors an examiner can try in a minute. The program is named msh. The binary is produced by Make from five C modules. Python is used only as a test runner. The shell itself links only the C library.

The rest of this report states the problem, compares msh with the shells it is modeled on, records the tools used to build it, explains the design and the implementation, and reports what the tests and the timings showed.

## 2. Problem statement and objectives

### 2.1 Problem statement

A description of fork and exec is easy to memorize and easy to forget, because nothing in the description can be typed. The same is true of a pipe. A pipe is a kernel buffer with two file descriptors. That sentence becomes concrete only when one process writes a descriptor and another process reads the other, and the reader sees the writer’s bytes. Redirection is the same idea pointed at an open file. Signals are the same idea pointed at a process group: the terminal sends an interrupt to whoever is in the foreground, and the shell has to arrange that “whoever” is the command, not the shell.

The problem is to build a command-line shell that performs these steps with its own system calls, and to show that the result is correct on ordinary input and on input that should fail. The shell must keep running after a failure. A missing program, a missing file, or a line that is only a pipe character is a normal event in interactive use. It must set a non-zero status and print a short error. It must not exit.

The shell must also be explainable. Every call that creates a process, a pipe, or a process group should sit in a module whose name matches the job. The report and the viva should be able to point at that module.

## 2.2 Objectives

1. Parse a command line into a pipeline, including quoted words, input redirection, output redirection, append redirection, and a background marker.
2. Execute external programs with fork and execvp, and collect their termination status with waitpid.
3. Execute cd, pwd, exit, help, history, jobs, and wait in the shell process when they are not part of a pipeline and not backgrounded.
4. Connect the stages of a pipeline with anonymous pipes, including pipelines of many stages, and close the unused ends so that readers observe end of file.
5. Apply file redirection in the correct stage: input on the first command, output on the last command, and both on a command that stands alone.
6. Place each pipeline in a new process group and, on a terminal, give that group the foreground so that Ctrl+C stops the command and the shell prints a new prompt.
7. Record background pipelines and reap them, so a finished background command does not remain a zombie, and so wait can return its status.
8. Accept a script on standard input with no prompt, which is what the automated tests use.
9. Include boundary and invalid cases in the test suite, and measure the wall-clock cost of a builtin, an external command, and a short pipeline.

## 2.3 Scope

msh implements the process and file-descriptor core of a shell. It does not implement the shell as a programming language. There is no if, while, case, or for. There is no command substitution, no globbing, and no general variables. The token $? is expanded when it is not quoted, and a word that is ~ or that starts with ~/ is expanded to the home directory. Those two expansions are enough to show that the shell, not the child program, rewrites the line. Job control stops at background and wait. Ctrl+Z does not suspend a process, and there is no fg or bg. Standard error is left on the terminal. A separate 2> redirection is outside the scope. These limits are deliberate. Each one is a feature that can be added later without changing the process model, and each one avoided is a feature the tests do not have to pretend to cover.

### 2.4 Expected outcome

The outcome is a working binary, a test suite that fails if a behavior regresses, a sample session, and the measurements in Section 7. A person who can type at a prompt should be able to run a command, a pipeline, a redirection, a background job, and an interrupt, and should be able to say which system call did the work.

### 2.5 Alignment with the prescribed topic

The course topic list, item 21, asks for a Mini UNIX Shell: a command-line interpreter that supports commands and process creation with fork() and exec() on Linux, written in C. msh is that program. External commands are children. The child calls execvp, which is the PATH-searching member of the exec family, so a typed name such as ls becomes a new program image. The parent waits with waitpid. Builtins that must change the shell, cd and exit, stay in the parent, because a child cannot change its parent’s directory or end the shell by exiting.

The same topic row does not require pipes, redirection, or background jobs. Those are included because the submission guidelines ask for a working command-line interface, invalid-input tests, and a measured result, and because a shell that can only run one command with no status and no Ctrl+C behavior cannot be demonstrated properly. Each extra feature uses the same process and file-descriptor model. None of it is delegated to another shell.

The prescribed syllabus places this work in process management and in the UNIX introduction: process creation, process state, and the services the operating system provides to a user program. The shell is the user program that calls those services. The assessment plan lists the mini-project under CO5, with a maximum of 5 marks, and a tentative date of 24 October 2026. The submission guidelines say to follow the deadline the instructor announces. The topic form for this class set topic submission at 25 October 2026, 00:00, and final submission at 2 November 2026, 16:00.

The guidelines also give a suggested 100-point rubric. The instructor may scale it to the 5-mark component.

Table. Suggested rubric from the submission guidelines

Criterion Marks Problem understanding and relevance to OS 10 Design, algorithm, and technical correctness 20 Implementation and working functionality 25 Testing, results, and

analysis 15 Report quality, structure, and references 15 Demonstration, viva, and individual contribution 15 Total 100

The demonstration the guidelines suggest is 8–10 minutes, then 3–5 minutes of questions. The order is the problem, the OS concept, the design, a live run, the test results, and the limitations. A backup recording of the same commands is in samples/session.txt if the lab machine cannot compile during the viva. The live binary is still the submission.

## 3. Literature and existing systems

### 3.1 The UNIX process model

Ritchie and Thompson's UNIX paper is the right starting point because the shell's features are consequences of the process model, not additions to it [1]. A process is created by duplicating an existing one. The new process can then replace its own program image. Parent and child share open file descriptors at the moment of the fork, which is why a shell can open a pipe, fork, and have both ends appear in the child. The paper also records that the system could start asynchronous processes, which is the background job in miniature, and that the command interpreter was a user program rather than a part of the kernel. msh follows that split. Nothing in this project is a kernel modification. The shell is an ordinary user process.

Modern textbooks organize the same facts into chapters a course already uses. Silberschatz, Galvin, and Gagne treat process creation, termination, and inter-process communication with message streams and shared memory as separate tools [6]. A pipe is a message stream with a fixed producer and consumer. Tanenbaum and Bos describe pipes the same way and discuss the shell as the classic client of fork and exec [7]. Arpaci-Dusseau's "Three Easy Pieces" is useful for the process-API view: the shell is the program that makes the API visible, and the cost of creating a process is something that can be measured rather than only described [8]. Section 7 of this report is that measurement.

### 3.2 Bourne's shell and what this project keeps

Bourne's 1978 paper describes the shell as two things at once [2]. As a command language it finds programs by searching directories, redirects input and output, and starts processes that communicate through pipes. As a programming language it has variables, control flow, and a return code that selects the next branch. The abstract of that paper is almost a requirements list for a shell project. msh keeps the command-language half and one piece of the return

code. Every pipeline records a status. $? expands to it. The programming-language half is named in the future-scope section and is not implemented, because a correct if statement is a parser project sitting on top of a shell, and the operating-system content is the layer underneath.

Kernighan and Pike show why the command-language half is worth getting right [3]. Filters become a toolkit only when the shell can bind the output of one to the input of the next. Their examples are pipelines of existing programs. msh can run those examples when the programs exist on the machine: sort, wc, cat, and printf are not built into msh. They are children. That choice keeps the shell honest. Re-implementing sort inside the shell would demonstrate an algorithm from a different unit.

### 3.3 What a production shell adds

bash and the POSIX shell command language are the existing systems a user already knows [9]. They add parsing rules that took decades to settle: word splitting, globbing, parameter expansion, here-documents, arithmetic, functions, and job control with suspension. Comparing msh with that list is useful because it shows the line between an operating-system demonstration and a product. Table 1 records the comparison that matters for this project. A dash means the feature is absent. A yes means the feature behaves as a user of a POSIX shell would expect for the cases in the test suite.

Table 1. Feature comparison

Feature Bourne / POSIX shell msh External command via PATH yes yes Builtin cd and exit yes yes Pipeline of n commands yes yes, up to 32 stages Input and output redirection yes yes, < > and >> Background with & yes yes, trailing & only Foreground Ctrl+C yes yes Ctrl+Z, fg, bg yes no &&, ||, and ; yes no Globbing and variables yes $? and ~ only Separate stderr redirection yes no Shell scripts with if/while yes no

The POSIX standard is the reference for the calls even where the language is smaller [9]. fork, exec, pipe, dup2, waitpid, setpgid, and tcsetpgrp are specified there. msh uses the C library wrappers for those calls and does not depend on a Linux-only interface. The same sources are intended to build with clang or gcc on a POSIX system. The measurements in Section 7 were taken on one machine, which is named there, and should be repeated if the binary is marked on a different kernel. The behavior tests do not depend on the timing numbers.

### 3.4 Process groups and the terminal

Stevens and Rago, and Kerrisk, both spend a chapter on sessions and process groups because this is the part students mis-implement first [4], [5]. A terminal has one foreground process group. The terminal driver sends the interrupt and quit signals to every member of that group. A shell that leaves its children in its own group will either die with them or, if it ignores the interrupt, will also see the children ignore it if they inherited the ignored disposition. The correct arrangement has three parts. The shell ignores the interrupt, or is simply not in the foreground group while a command runs. Each child resets the interrupt to the default action before exec. The child pipeline is placed in a new process group, and that group is moved to the foreground with tcsetpgrp for the duration of the command. When the command exits, the shell takes the foreground back.

Background commands are the complementary case. They must not be in the foreground group, or Ctrl+C would kill them too. msh gives every pipeline, foreground or background, its own process group. Only a foreground pipeline receives the terminal. This is the design in Section 5.4, and it is the behavior checked by the terminal test in Section 7.

### 3.5 Why system() is the wrong tool

The C library provides system(), which passes a string to the platform shell. Using it would make msh a thin prompt in front of the shell the project is supposed to be. It would also make the status, the pipeline, and the signal behavior belong to that other shell. The implementation section states the rule the code follows: the string the user typed is never passed to system() or to exec of another shell. argv arrays are built by the parser and handed to execvp. That is also why quotes are handled in msh. The kernel does not know that two words were inside one pair of quotes. The shell has to.

## 4. Requirements and tools

### 4.1 Functional requirements

R1. The shell reads lines from standard input until end of file or until exit. R2. On a terminal it prints a prompt of the form msh:$ on standard error. A home-directory prefix is shown as ~. R3. An external command runs as a child. If execvp cannot find it, the child exits 127 and the shell prints “command not found”. R4. cd, pwd, exit, help, history, jobs, and wait are

builtins. A simple builtin runs in the shell. cd changes the shell’s working directory. exit ends the shell. R5. A pipeline cmd1 | cmd2 | cmd3 runs each stage in its own process, with standard output of each stage except the last attached to standard input of the next. R6. < file opens file for reading on standard input. > file creates or truncates file for standard output. >> file appends. R7. A trailing & runs the pipeline in the background. The shell prints a job id and a process-group id and does not wait. jobs lists running jobs. wait waits for them. R8. $? expands to the status of the previous pipeline when the token is not quoted. R9. Ctrl+C on a terminal stops the foreground pipeline and returns to the prompt. A background sleep started just before that command is still listed by jobs. R10. Invalid input produces an error on standard error and a non-zero status, and the shell accepts another command. The invalid cases required here are an empty stage in a pipeline, a redirection with no target, a duplicate redirection, an unclosed quote, an ampersand that is not last, a missing input file, a directory cd cannot enter, and a non-numeric exit status.

### 4.2 Non-functional requirements

The source is C11 and is split by responsibility so that a reader can open the parser without reading the job table. Names of functions match the action: parse_line, execute_pipeline, builtin_run, jobs_reap. Errors use the prefix msh: and the system error string when the failure is a system call. The project builds with warnings enabled, including pedantic warnings. The test suite is one command, make test. No network access and no extra packages are required to build or to test.

### 4.3 Users and operating environment

The user is a person at a terminal, or a test harness that writes a script to the shell’s standard input. The shell is a foreground process of that terminal when it is interactive. It is not a login facility and it is not installed setuid. It should not be installed setuid later without a different design, because a setuid shell has to clean its environment before exec. That problem is out of scope and is stated so it is not mistaken for an oversight.

### 4.4 Tools

Table 2. Tools

Tool Role C11 implementation language, as specified for topic 21 POSIX C library fork, execvp, pipe, dup2, waitpid, setpgid, tcsetpgrp, signals Linux demonstration operating system

named by the topic list; the OS laboratory uses Fedora cc gcc on Fedora, or clang; the measurement binary was built with Apple clang 21.0.0 Make build, test, benchmark, and pack targets Python 3 functional tests, the Ctrl+C test, and the benchmark driver POSIX terminal manual use and the pseudoterminal used by the Ctrl+C test

The measurement machine reported Darwin 27.0.0 on arm64. The pipe-capacity probe and the timings in Section 7 were collected there. Functional tests were run on the same machine against the binary produced by make. The sources use POSIX calls only, so the same Makefile is the build on Fedora. Re-run make bench on the demonstration machine and replace Table 6 if the viva is marked from that machine. Do not present the Darwin numbers as Fedora numbers.

### 4.5 Deliverables

The deliverables are the source, the Makefile, the test suite, the sample session in samples/session.txt, this report, and a public README that describes how to build and run the program. The README does not carry roll numbers or course identifiers. Those belong on the cover of the printed report and on the topic form.

## 5. System design

### 5.1 Architecture

msh is a loop around four modules. Figure 1 shows the path of one line.

Figure 1. Path of one command line

stdin | v main loop —- history record | v parser —- Pipeline of Command records | +– simple builtin? – yes –> builtin module, optional redirected fds | no | v executor |– pipe() between stages |– fork() one child per stage |– setpgid() so the pipeline shares a process group |– tcsetpgrp() if this is a foreground terminal command |– waitpid() the group, or jobs_add() if background v status stored for $?

The main loop owns the prompt, the signal disposition of the shell, and the decision to exit. The parser owns syntax. It does not fork. The executor owns processes and file descriptors. The builtin module owns operations that must affect the shell process. The job module owns the table of background process groups and the reaping of zombies. Keeping fork out of the

parser means a syntax error cannot leak a process. Keeping cd out of a careless fork means a directory change is visible to the next command.

### 5.2 Data structures

A Command holds a NULL-terminated argv array, an optional input path, an optional output path, and a flag for append. A Pipeline holds up to 32 commands and a background flag. Limits are fixed so the report can state them and the tests can approach them: 128 arguments per command, 32 stages per pipeline, 64 background jobs, and 100 history lines. Hitting a limit prints an error and does not overflow the array.

The parser allocates each word with malloc. The executor borrows the pipeline for the duration of the command and the main loop frees it afterward. History keeps its own copy of the raw line. A background job keeps its own copy of the raw line, the process-group id, and the pid of each stage, together with a flag that says whether that pid is still alive. When the last pid of a job has been reaped, the job is removed and its status is held so that a later wait can return it. This matters for a script that writes `false &` and then `wait`. The child may exit before wait is parsed. The shell must already have reaped it, or it would be a zombie, and must still remember the status 127.

### 5.3 Parsing

The parser is a single left-to-right scan. Whitespace separates words. A word runs until whitespace or one of the operators |, <, >, and &, except that a single or double quote toggles a quoted region in which those characters are ordinary data. Adjacent quoted and unquoted pieces form one word, so ‘a’“b” is the argument ab. An unclosed quote is a syntax error. A # outside quotes starts a comment that runs to the end of the line. A line whose first non-whitespace character is # is ignored by the main loop before parsing, and it is not stored in history.

The operator | finishes the current command and starts the next, provided the current command has a program name. A leading pipe, a trailing pipe, or a doubled pipe is a syntax error (“empty command” or “missing command”). The operator < consumes the next word as the input path. The operator > consumes the next word as the output path, and >> sets the append flag. A second input redirection on the same command, or a second output

redirection, is a syntax error. The operator & is legal only as the last token. It sets the background flag on the pipeline.

After the scan, every stage must have at least one argument. A line that is only `> outfile` is rejected. The shell could have created the file, which is what a large shell does, but a missing program name is a more honest error for a project of this size, and it is easy to demonstrate.

Expansion happens on unquoted words only. The word $? is replaced with the decimal status of the previous command. A word equal to ~, or starting with ~/, is prefixed with the value of HOME. Quoted $? stays the two characters dollar and question mark, which the tests check. ~user is not looked up in the password database.

### 5.4 Execution and file descriptors

If the pipeline has one command, that command is a builtin, and the pipeline is not backgrounded, the executor runs the builtin in the shell. Redirection of a builtin is done by duplicating the old standard input or output, dup2-ing the file onto descriptor 0 or 1, running the builtin, flushing stdio, and restoring the saved descriptors. The flush is required because the C library buffers a FILE object. Writing the buffer after the descriptor has been restored would send the builtin’s output to the terminal instead of the file. The same flush is required before a builtin in a child calls _exit, because _exit does not flush stdio. exit of the C library would flush, but it would also run handlers that belong to the shell, so the child flushes and then calls _exit.

Every other pipeline is a sequence of forks. The parent creates n-1 pipes for n stages before forking, so every child inherits the pipe descriptors it needs. Child i duplicates the read end of pipe i-1 onto standard input when i is not zero, and the write end of pipe i onto standard output when i is not the last stage. It then closes every pipe descriptor that is not 0 or 1. Closing is the difference between a pipeline that finishes and a pipeline that hangs. The reader of a pipe sees end of file only when every write end is closed. If the shell, or a later stage, kept a write end open, wc would wait forever.

File redirection is applied after the pipe duplication and overrides it for the ends that the syntax attached to a stage. Input redirection is applied to stage 0. Output redirection is applied to the last stage. So `printf 'b\na\n' | sort > out` leaves sort writing to out, not to a pipe. A redirection open that fails prints the path and the system error and the child exits 1. The parent still waits, so the failure becomes the pipeline status.

The child then either runs a builtin and _exits with its status, or calls execvp. A missing program exits 127. A program that exists but cannot be executed exits 126. The parent never treats exec failure as its own errno, because exec failure happens in the child after the fork.

The status of a foreground pipeline is the status of the last stage, which matches the rule users know from a POSIX shell [9]. A child that dies from signal N is reported as 128+N. An interrupt is signal 2 on this platform, so Ctrl+C on sleep yields 130. The shell prints a newline in that case so the next prompt does not sit on the ^C line.

### 5.5 Process groups

Figure 2 shows the groups while a foreground pipeline runs.

Figure 2. Foreground pipeline

terminal foreground group | v +———————————+ | process group = pid of | | the first child | | child 0 —- pipe —- child 1 | +———————————+

shell process group (not foreground, SIGINT ignored as a second line of defense)

The first child becomes the process-group leader. The parent and the child both call setpgid with that same id. Calling it on both sides avoids a race in which the child runs before the parent has put it into the group. Later children join the same group. The shell then calls tcsetpgrp to make that group the foreground, waits until every member has been reaped, and calls tcsetpgrp to take the terminal back.

The shell itself ignores SIGINT, SIGQUIT, and SIGTSTP. Children set SIGINT and SIGQUIT back to the default action, so a program that does not catch Ctrl+C dies. SIGTSTP is left ignored in the child as well. If it were restored to the default, Ctrl+Z would stop the child while the shell was blocked in waitpid without the WUNTRACED flag, and the prompt would not return. Suspension is future work. Ignoring the signal is the choice that keeps the shell responsive. SIGPIPE is restored to the default in the child so a stage that writes into a pipe whose reader has exited can be killed by the kernel, which is what makes `yes | head` able to end if a user types it.

Background pipelines use the same fork and setpgid path and skip tcsetpgrp and the wait. The shell stays in the foreground. Ctrl+C therefore does not reach the background group. The terminal test starts `sleep 30 &`, starts a second `sleep 30` in the foreground, sends Ctrl+C, and

checks that jobs still lists the background sleep. That is the observation that shows the two groups are distinct. A shell that put every child in its own group and forgot tcsetpgrp would fail the other way: Ctrl+C would hit the shell’s group, the shell would ignore it, and the foreground sleep would keep running. A shell that left children in the shell’s group would kill the background sleep as well. The test is aimed at both failures.

### 5.6 Jobs and zombies

A child that exits while its parent does not wait becomes a zombie. The main loop calls jobs_reap before each prompt, using waitpid with WNOHANG, and removes jobs whose members have all exited. On a terminal the shell prints a done line to standard error. In a script it stays quiet, because a test’s standard output should be the command’s output, but it still remembers the status. wait walks any jobs that are still running, blocks in waitpid for each remaining pid, and returns the held status. If nothing is running and nothing has finished since the previous wait, wait returns 0.

jobs prints one line per running job: the job id, the word running, the process-group id, and the original line. The process-group id is the pid a user can pass to the external kill command. msh does not implement a kill builtin.

### 5.7 Builtins

cd with no argument changes to HOME and reports an error if HOME is unset. cd - changes to OLDPWD and prints the new directory. Any other argument is passed to chdir. On success the shell updates the PWD and OLDPWD environment variables so that a child sees them. cd with more than one argument is an error and does not change directory. cd of a file fails with the system error “Not a directory”.

pwd prints the result of getcwd. exit with no argument exits with the previous status. exit with one integer from 0 to 255 exits with that value. Any other argument prints an error and does not exit, so a typo cannot end a session the user meant to keep. help prints a short summary of the syntax. history prints the stored lines, including the history command itself, up to 100 entries.

A builtin inside a pipeline cannot run in the parent. `cd /tmp | cat` forks, and the chdir happens in the child, which then exits. The shell’s directory is unchanged. This is the correct behavior,

and a test locks it in. The same rule is why exit inside a pipeline does not exit msh. The child calls _exit.

### 5.8 Signals at startup

If standard input is a terminal, the shell attempts to place itself in its own process group and to take the foreground with tcsetpgrp. A failure to take the terminal clears the “owns the tty” flag. The shell still prints prompts and runs commands, but it does not call tcsetpgrp around children. That fallback is what a non-interactive script uses as well: isatty is false, there is no prompt, and there is no foreground switching. The test suite depends on this. A test that inserted prompts into standard output would be comparing the prompt, not the command.

### 5.9 Error status summary

Table 3. Status values msh produces

Condition Status Successful command 0, or whatever the command returned Builtin failure such as a bad cd 1 Redirection open failure 1 Syntax error 2 Program found but not executable 126 Program not found 127 Process killed by signal N 128+N Background launch accepted 0, until wait reports the job

## 6. Implementation

### 6.1 Files

Table 4. Source modules

File Lines (approx.) Responsibility src/main.c 126 prompt, getline loop, signal setup src/parser.c 271 scan, quotes, syntax errors, expansion src/execute.c 293 pipes, fork, redirection, process groups src/builtin.c 163 cd, pwd, exit, help, history src/jobs.c 162 job table, reap, wait headers 86 shared structs and prototypes

The Makefile compiles with -std=c11 -Wall -Wextra -Wpedantic. The binary is ./msh. make test and make bench are the two commands used in Section 7. A separate probe, tests/measure_pipe.c, writes to a non-blocking pipe until the kernel returns EAGAIN and prints how many bytes were accepted. That number is the pipe capacity on the host, which is an operating-system parameter rather than a constant in msh.

### 6.2 Main loop

getline reads an arbitrary line. The trailing newline is stripped, and leading and trailing whitespace is removed. An empty line or a comment returns to the prompt without changing the status. Any other line is copied into the history buffer before parsing, including lines that will turn out to be syntax errors, because the user typed them. parse_line either fills a Pipeline or prints a syntax error and returns failure, in which case the status becomes 2. execute_pipeline returns the status that $? should see. If exit was requested, the loop ends and the process returns that code to its parent. End of file is the same exit path without an explicit exit. The status on end of file is the status of the last command, so a script whose last command is false makes msh itself exit 1.

### 6.3 Parser details that the tests rely on

The word reader allocates a buffer the size of the whole line, which is enough because a word cannot be longer than the line. Quotes are not copied into the word. A backslash inside double quotes is an escape for a quote, a backslash, or a dollar sign, and is otherwise kept. The expansion of ~ copies HOME and the remainder of the word into a new buffer and frees the old one. Failures to allocate abort the shell with a short message. A shell that cannot allocate a word cannot recover usefully, and hiding the failure would make later crashes harder to explain.

Syntax errors are printed from the parser so the executor can assume that a pipeline it receives is well formed: every stage has an argv[0], redirection paths are non-NULL when the flags say they exist, and at most one input and one output path are set.

### 6.4 Executor details

Pipe descriptors are stored in one array of length 2*(n-1). If pipe fails halfway, the descriptors already opened are closed and the function returns 1 without forking. If fork fails halfway, the process group already started is sent SIGTERM and reaped, and the function returns 1. The child path is written so that every error ends in _exit rather than returning into the parent’s loop. A child that returned would become a second shell reading the same input.

dup2 of a pipe end onto descriptor 0 or 1 is skipped when the pipe end is already that descriptor, and the later close loop does not close descriptor 0 or 1. This avoids the classic mistake of closing standard input because a pipe happened to be allocated on descriptor 0,

which can happen if the shell was started with a closed standard stream. The tests do not close standard input, but the guard is small and the bug it prevents is silent.

setpgid errors in the parent are ignored when the child has already performed the same call. The child may have exited by the time the parent runs, which returns ESRCH on some kernels. The group is still correct because the child set it first.

### 6.5 Builtins and the environment

chdir is the whole of cd. The shell does not resolve “..” itself. getcwd after a successful chdir supplies the path stored in PWD, which is the physical path. On the test machine, cd /tmp followed by pwd prints /private/tmp, because /tmp is a symbolic link. The tests compare against pwd -P for that reason. A test that expected the string /tmp would be testing the symlink, not the shell.

exit does not call the C library exit() from inside the builtin when the builtin is running in the parent. It sets a flag that the main loop observes after redirection has been restored. Calling exit() immediately would skip that restoration and would skip freeing the line buffer. In a child, exit must not run the parent’s handlers, so the builtin calls _exit directly.

### 6.6 What was intentionally left out of the code

No global search path is implemented by hand. execvp already searches PATH, and reimplementing it would duplicate a library routine without teaching a new idea. No command is executed by passing the raw line to /bin/sh. The history buffer is a fixed ring of strings, not a file, so it does not survive exit. The job table does not survive exit either. Background processes are not signaled when the shell exits. A production shell often sends SIGHUP. msh lets them be reparented to init. For the demo, the terminal test kills the background sleep it started, so the test does not leave a sleep 30 behind.

## 7. Testing and results

### 7.1 Method

Functional tests are a Python unittest module that starts ./msh, writes a script to its standard input, and compares standard output, standard error, and the process exit code. The shell is not a terminal in these tests, so the prompt must not appear, and tcsetpgrp must not be

required. Each test uses a fresh process. Temporary files are created in a temporary directory that the test deletes. Paths passed to the shell contain no spaces, so the tests are not accidentally depending on quote removal except where a test is specifically about quotes.

The interrupt test is separate because it needs a terminal. Python’s pty.fork gives the shell a pseudoterminal and makes the shell a session leader, which is the situation msh has when a person opens it in a terminal window. The test writes sleep 30 &, then sleep 30, then the byte Ctrl+C, then jobs, then printf SURVIVED, then exit. It checks three things: the prompt comes back after Ctrl+C, jobs still shows the background sleep, and SURVIVED is printed. The background pid is then killed so the test cleans up after itself.

### 7.2 Functional results

All 36 functional tests passed, and the interrupt test passed, on the measurement machine after a clean build. Table 5 groups them by the requirement they lock down. Figure 3 is a session captured from the terminal: echo, a two-stage pipeline through sort, and $?. The prompt returns after each command. The same commands are in samples/script.txt for a non-interactive run.

Figure 3. Captured msh session




*Captured msh session*

Table 5. Functional coverage

Group What was checked Tests Simple execution echo, quotes, adjacent quotes, comments, help, history 6 Directory builtins pwd, cd, cd -, tilde, missing path, file path, 8 too many arguments, HOME unset, cd inside a pipeline Redirection create, append, input, empty input, missing input, 5 builtin pwd redirected to a file Pipelines sort, three stages, 31-stage cat

chain, output file, 6 status of the last stage, builtin pwd flushed through a pipe Errors command not found (127), six syntax forms (status 2), 4 duplicate redirection, bad exit argument Process lifetime exit code, exit of previous status, EOF status, 5 background sleep with wait, background command not found Interrupt Ctrl+C kills foreground sleep, background sleep remains 1

The six syntax forms are: a trailing pipe, a leading pipe, a redirection with no target, an unclosed quote, an ampersand followed by another word, and a redirection with no command. Each leaves the shell able to print $? as 2.

Representative observed results, copied from a run:

- echo "quoted args" prints quoted args.
- cd /tmp then pwd prints /private/tmp on the measurement machine.
- echo one > file then echo two >> file then cat < file prints one and two.
- printf 'c\nb\na\n' | sort | wc -l prints a right-aligned 3. The spaces come from wc.
- false then echo $? prints 1.
- no_such_cmd prints msh: no_such_cmd: command not found and $? is 127.
- ls | prints msh: syntax error: missing command and $? is 2.
- sleep 1 & then jobs prints [1] running pgid=<pid> sleep 1 & and a line [1] <pid> on standard error. wait then completes and the shell can print the next command.

Boundary cases are the empty input file, the missing input file, cd to a file, cd with HOME removed from the environment, a non-numeric exit, a quoted $?, and the 31-stage pipeline. The long pipeline is the descriptor test. Thirty cat processes each inherit every pipe if the parent or the children forget to close an end. The command echo hello followed by thirty | cat stages prints hello and exits 0. A leaked write end would make the test hang until the timeout.

### 7.3 Measurements

Timings are wall-clock measurements around a single msh process that runs the same command many times. Starting the shell once means the cost of process creation inside the shell is what the numbers show, not the cost of starting msh itself. Each figure is the median of five trials, divided by the number of commands. The benchmark driver is tests/benchmark.py, run by make bench.

Table 6. Median time per command

Workload Commands per trial Median per command pwd > /dev/null 400 0.024 ms /usr/bin/true 200 1.244 ms echo hello | cat > /dev/null 100 1.823 ms

The five trial totals, in seconds, were:

- builtin, 400 commands: 0.0096, 0.0095, 0.0093, 0.0094, 0.0094
- external true, 200 commands: 0.2513, 0.2500, 0.2444, 0.2488, 0.2484
- pipeline, 100 commands: 0.1847, 0.1828, 0.1823, 0.1821, 0.1815

pwd was chosen because it does a single getcwd and a write, and the write was sent to /dev/null so the terminal was not part of the cost. It still has to be a builtin. If pwd had been /bin/pwd, the number would have included a fork. /usr/bin/true was chosen because the program does no work. The 1.244 ms is almost entirely fork, exec, and wait. The builtin is about 52 times cheaper on this machine (1.244 / 0.024). That ratio is the practical reason shells implement cd, pwd, and exit as builtins. An external cd would change the directory of a child that immediately exits, which is both slower and useless. The pipeline costs more than one external command because it creates two processes and one pipe, then waits for both. It costs less than two independent true processes would, which is consistent with the two children running rather than with the shell doing their work twice in series, but the benchmark was not designed to split that overlap out. The claim the numbers support is the smaller one: a builtin avoids a process, and a pipeline pays for more than one process.

Table 7. Pipe capacity

Measurement Value Bytes accepted by a non-blocking 65536 write before EAGAIN

The probe sets the write end non-blocking and writes 4096-byte buffers until the write would block. 65536 bytes is the capacity of an empty pipe on this kernel. It is not compiled into msh. The relevance to the shell is the reason a pipeline can run two processes at once. The writer runs until the kernel buffer is full. If the reader is running, it frees space and the writer continues. If the reader is not running, the writer blocks inside write, which is the kernel scheduling the producer behind the consumer. msh does not implement a user-space buffer between stages and does not need to. The pipe is the buffer.

### 7.4 What was not measured

CPU time was not split into user and system time. The wall clock is the quantity a person at the prompt feels, and it is the quantity the benchmark records. Memory use of msh itself is small: a leak check of a short script reported 0 leaks for 0 leaked bytes, with a physical footprint of about 2.6 MB, most of which is the process image and the C library rather than the job table. That check is a sanity test, not a performance result. The shell was not compared with bash on the same scripts. bash does more work per line, so a lower number for msh would not mean msh is a faster shell in general. It would mean msh does less.

## 8. Discussion

The design choice that matters most is the decision to run simple builtins in the parent and everything else in a child. It is also the choice that is easiest to get wrong in one direction or the other. Running every builtin in a child makes cd a no-op. Running every builtin in the parent makes `pwd | cat` write to the terminal and makes `cd /tmp | cat` change the shell as a side effect of a pipeline. The rule “in the parent only when the builtin is the whole pipeline and is not backgrounded” handles both, and the tests cover both. The stdio flush belongs to the same decision. The kernel file descriptor and the C library buffer are different objects. Redirecting descriptor 1 does not redirect bytes that printf has already stored and not written. Flushing before the descriptor is restored, and flushing before _exit, is part of implementing redirection correctly for builtins.

The second choice is the process group. A shell that only calls signal(SIGINT, SIG_IGN) appears to survive Ctrl+C in casual testing, because the shell ignores the signal, but the children inherit the ignored signal unless they reset it, and a background job in the same group dies or survives together with the foreground job. The terminal test is specific because a weaker test, “the shell printed another prompt”, can pass for a broken process-group implementation. Requiring the background sleep to survive is what makes the test about groups rather than about a signal mask.

The third choice is to reject a few convenient syntaxes. Semicolons, &&, and globs would make demonstrations prettier. They would also move work into the parser that the operating-system part of the project does not need. The sample session is still a credible shell session: quotes, a directory change, a pipeline into wc, a missing program, and a syntax error. That is enough to show the system calls.

A limitation that should be said plainly in a viva is the ignored SIGTSTP. The shell does not implement suspension. Ctrl+Z does nothing useful. Implementing it correctly means waiting with WUNTRACED, recording a stopped job, and continuing it with SIGCONT under fg. The data structure for a job is already close to what that feature needs. The missing piece is the terminal handover on fg, which is the same tcsetpgrp path the foreground pipeline already uses.

Another limitation is that standard error of a child is the shell's standard error. `cmd > file` saves standard output only. Diagnostic lines from the child still appear on the terminal. That matches the requested scope and surprises people who expect `>` to mean "all output". The report states it here so the demo can use it as a talking point rather than trip over it.

The measurements should not be over-read. They show that process creation dominates an empty command on one machine. They do not show that pipes of 65536 bytes are universal. Linux has used 65536 as a default as well, but a different kernel, or a pipe whose capacity an administrator has changed, will produce a different probe result. The right way to cite the number is "on the test machine, the probe observed 65536 bytes."

# 9. Conclusion and future scope

## 9.1 Conclusion

msh is a command-line UNIX shell that implements the process, pipe, redirection, and foreground-group behavior required of a small shell. External commands are children created by fork and execvp. Builtins that change the shell run in the shell. Pipelines are children connected by anonymous pipes, with unused ends closed. Redirection is dup2 of an open file onto standard input or standard output. Background commands are tracked and reaped. On a terminal, Ctrl+C stops the foreground pipeline and does not stop a background job, and it does not stop the shell. Invalid lines produce a status and an error and leave the shell running. The test suite locks these behaviors in, and the timings show why a builtin exists: creating a process to do nothing costs about 1.2 ms on the test machine, and avoiding that creation costs about 0.02 ms.

The project meets the objectives in Section 2. The parts that were left out are named in Section 2.3 and below, and they are left out as scope, not as unfinished versions of the same features.

### 9.2 Future scope

A following version could add fg and bg, with stopped jobs and SIGCONT, reusing the job table. It could add &&, ||, and ; as list operators around the pipeline type that already exists. It could add globbing with the POSIX glob function on unquoted words, and a separate stderr redirection. None of those changes requires a different process model. A larger change, and a different project, would be to add variables and the if/while constructs Bourne described [2]. That would be a language interpreter whose system-call layer is the shell this report already built.

References

[1] D. M. Ritchie and K. Thompson, “The UNIX time-sharing system,” Commun. ACM, vol. 17, no. 7, pp. 365–375, Jul. 1974, doi: 10.1145/361011.361061.

[2] S. R. Bourne, “UNIX time-sharing system: The UNIX shell,” Bell Syst. Tech. J., vol. 57, no. 6, pp. 1971–1990, Jul.–Aug. 1978, doi: 10.1002/j.1538-7305.1978.tb02139.x.

[3] B. W. Kernighan and R. Pike, The UNIX Programming Environment. Englewood Cliffs, NJ, USA: Prentice-Hall, 1984.

[4] W. R. Stevens and S. A. Rago, Advanced Programming in the UNIX Environment, 3rd ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2013.

[5] M. Kerrisk, The Linux Programming Interface. San Francisco, CA, USA: No Starch Press, 2010.

[6] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed. Hoboken, NJ, USA: Wiley, 2018.

[7] A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed. Upper Saddle River, NJ, USA: Pearson, 2015.

[8] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, Operating Systems: Three Easy Pieces. Madison, WI, USA: Arpaci-Dusseau Books, 2018. [Online]. Available: https://pages.cs.wisc.edu/~remzi/OSTEP/

[9] IEEE and The Open Group, The Open Group Base Specifications Issue 7 (IEEE Std 1003.1-2017), 2018. [Online]. Available: https://pubs.opengroup.org/onlinepubs/9699919799/

[10] M. J. Bach, The Design of the UNIX Operating System. Englewood Cliffs, NJ, USA: Prentice Hall, 1986.

The course syllabus for Operating Systems (T7998) prescribes Silberschatz, Galvin, and Gagne, 9th edition; Stallings, 8th edition; Tanenbaum and Bos, 4th edition; Godbole, 2nd edition; and Bach [10]. References [6] and [7] are the later or matching editions of two of those books. Reference [6] is the 10th edition of the Silberschatz text, used because that is the edition consulted while writing this report. The 9th edition is the prescribed one.

Appendix A. Sample session

The transcript in samples/session.txt is the one to attach or to redraw if the printed report needs a figure of the screen. It includes help, a quoted echo, cd to /tmp, cd -, append redirection, a three-stage pipeline, a failing command, an unknown command, and a syntax error. It also includes the background form [1] running pgid=<pid> sleep 1 &. Process ids in that file are from one run and will differ on another run.

Appendix B. Test catalogue

The executable catalogue is tests/test_shell.py and tests/test_signal.py. Running make test is the check that this appendix describes. The groups are the rows of Table 5. A marker should treat a failure of make test as a failure of the corresponding requirement in Section 4.1, not as a failure of the test harness. The tests do not require a network, a display, or a privileged account. The interrupt test requires permission to allocate a pseudoterminal, which a normal user has.

Appendix C. Module map for the viva

Question: where is a pipeline created? Answer: parse_line in src/parser.c fills a Pipeline. It does not create processes.

Question: where is a process created? Answer: execute_pipeline in src/execute.c calls fork, then execvp in the child.

Question: why does cd work? Answer: a simple cd is recognized in execute_pipeline and run by builtin_cd through chdir in the shell process. The same cd as part of a pipeline runs in the child, and a test checks that the shell directory does not change.

Question: why does Ctrl+C not kill msh? Answer: the foreground pipeline is a different process group, and tcsetpgrp makes that group, not the shell, the foreground of the terminal. Children reset SIGINT to the default action. The shell ignores SIGINT as a backup.

Question: why does the last command decide $? Answer: wait_pipeline records the status of the last pid in the pipeline. Earlier children are still reaped, so they do not become zombies, but their statuses are not the pipeline status.

Question: what did the timing show? Answer: on the test machine, a builtin pwd was about 0.024 ms and /usr/bin/true was about 1.244 ms. The difference is the cost of creating a process that does no other work.